

AI Agent Orchestration Platform

Technical Documentation

Local-first multi-agent workflows with LangGraph, async execution,
Telegram messaging, and full observability.

Author: Rajaram

Version 1.0 | May 2026

Yuno AI Engineer Hiring Challenge

Version: 1.0

Repository: AI-Orchestrator

Purpose: Yuno AI Engineer hiring challenge submission

A local-first platform for creating AI agents, wiring them into multi-agent workflows, and running them with a real **LangGraph** runtime - not a UI mock. Built for the Yuno AI Engineer hiring challenge: agents collaborate asynchronously, conversation history is persisted, and at least one agent can talk to humans over **Telegram**.

Contents

1. [What you get](#)
2. [Quick start \(Docker\)](#)
3. [Architecture](#)
4. [Why LangChain / LangGraph](#)
5. [ReAct agents \(Reason + Act\)](#)
6. [Getting started](#)
7. [Telegram setup](#)
8. [Project structure](#)
9. [License](#)

What you get

- **Agent management** - CRUD, system prompts, models, tools, memory/schedules/guardrails, channel bindings
 - **Visual workflows** - React Flow canvas with conditional edges and revise loops
 - **Templates** - Research -> Writer; Planner -> Executor -> Reviewer (with feedback loop)
 - **Async execution** - API enqueues runs on Redis; a background worker executes LangGraph graphs
 - **Observability** - Run steps, SSE/live events, token counts, estimated cost
 - **Messaging** - Internal run transcripts plus Telegram (long polling, no ngrok required)
-

Quick start (Docker)

Minimum steps to run the full stack locally in **one command**:

1. One-time environment setup

Create a `.env` file in the project root with at least:

```
OPENAI_API_KEY=sk-...
```

`api` and `worker` load this via `docker-compose.yml`. The stack may start without a key, but LLM workflow runs and Telegram replies need it.

2. Start the stack

From the repo root:

```
make up
```

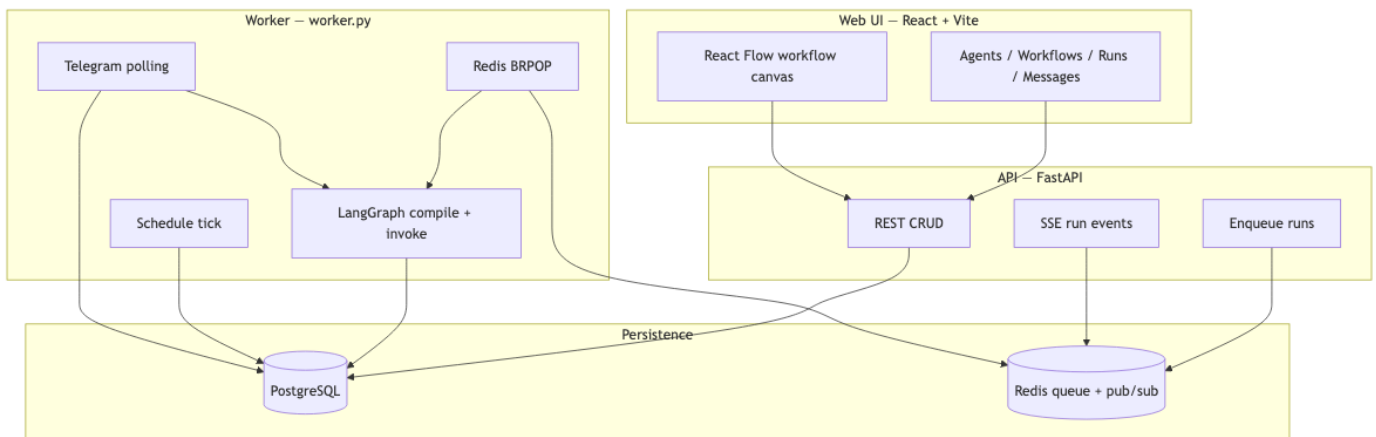
Same as `docker compose up --build -d`. This builds images, starts Postgres, Redis, API, worker, and frontend; runs migrations; and seeds templates/agents when the database is empty.

Service	URL
Web UI	http://localhost:5173
API	http://localhost:8000
OpenAPI	http://localhost:8000/docs

Stop with `make down`. More env options, Telegram, and non-Docker setup are covered in [Getting started](#).

Architecture

The system splits into four layers. The API never blocks on long LLM calls; the worker owns graph execution.



Layer	Location	Responsibility
Web UI	frontend/	Manage agents, build workflows, monitor runs, read message history
API	backend/app/api/	REST + OpenAPI; enqueue runs; stream run events
Runtime	backend/app/runtime/	Compile workflow JSON -> StateGraph; execute with ReAct agent nodes
Persistence	backend/app/models/	Agents, workflows, runs, steps, messages
Worker	backend/worker.py	Dequeue runs, run graphs, optional Telegram + scheduled jobs

Workflow execution path

1. User saves a workflow definition (nodes + edges) from the UI or a template.
2. Starting a run creates a Run row and pushes its ID onto `orchestrator:runs` in Redis.
3. The worker pops the ID, loads agents referenced by `node agent_id`, and calls `compile_workflow()` in `compiler.py`.
4. Each **agent node** runs a **ReAct** subgraph (`create_react_agent`), appends output to `shared_context`, and persists messages via callbacks in `executor.py`.
5. **Conditional edges** (e.g. `reviewer -> executor` when output contains "revise") are routed by `_condition_router` on `last_agent_output`.
6. Completed runs store token usage and estimated cost on the Run record.

Workflow JSON shape

Workflow definitions are stored in PostgreSQL:

```
{
  "nodes": [
    { "id": "planner", "type": "agent", "agent_id": "<uuid>", "is_entry": true },
    { "id": "end", "type": "end" }
  ],
  "edges": [
    { "source": "reviewer", "target": "executor", "condition": true, "label": "revise" }
  ]
}
```

For deeper extension notes (templates, new channels, cost tracking), see [docs/architecture.md](#) in the repository.

Why LangChain / LangGraph

This project sits on the **LangChain ecosystem** (models, messages, tools) and uses **LangGraph** for orchestration - the layer that turns your workflow diagram into executable control flow.

Need	How LangGraph helps
UI graph -> runtime	Workflow JSON maps directly to StateGraph nodes and edges, including conditional branches
Multi-agent handoff	Shared WorkflowState (messages, task, shared_context, last_agent_output) passes context between nodes
Tool-using agents	langgraph.prebuilt.create_react_agent gives a battle-tested Reason-Act loop per agent node
Production habits	Checkpoint-friendly state, explicit recursion limits, clear separation of compile vs invoke
vs CrewAI	Less opinionated "roles"; easier to reflect arbitrary graphs from the visual builder
vs AutoGen	Fits a simple API + worker + queue deployment without a separate agent runtime cluster

LangChain supplies ChatOpenAI, BaseMessage, and @tool definitions in [app/runtime/tools.py](#). LangGraph owns **when** each agent runs and **where** control goes next.

ReAct agents (Reason + Act)

Each agent node (and Telegram replies) uses a **ReAct** pattern: the model alternates between reasoning in natural language and calling tools until it produces a final answer. Implementation: `create_react_agent(llm, tools)` in `compiler.py` and `telegram.py`.

What makes our ReAct setup distinctive

Per-agent tool sets - Tools are selected from a registry (calculator, write_note, fetch_url_summary) based on the agent's tools JSON field, not a global bundle.

Workflow-aware prompts - Before invoke, the system prompt is augmented with: - shared_context from prior agents in the same run - The run's input_task as the user goal - Up to the last 10 messages from persisted history

Guardrails - Agent config.guards can cap max_output_chars and influence max_iterations / graph recursion_limit in the executor.

Observable outputs - An on_agent_output callback persists each node's reply as a Message and emits Redis events for the live run UI - so ReAct tool steps are not invisible black boxes at the workflow level (final node output is what gets summarized per step).

Same brain in Telegram - Bound agents use the same ReAct executor and tools as in workflows, with transcripts stored under channel=telegram.

Conditional routing from agent text - The reviewer template can say "revise"; _condition_router reads last_agent_output and routes the revise edge back to the executor without hard-coding tool names.

ReAct is ideal here because roles like Researcher and Executor need **optional** tool use (fetch URLs, math) inside a fixed graph, while LangGraph still controls **order** and **loops** between agents.

Getting started

See [Quick start \(Docker\)](#) for the shortest path (make up). This section adds full configuration detail.

Prerequisites

- [Docker](#) and Docker Compose
- An [OpenAI API key](#) (required for live LLM runs)
- Optional: [Telegram bot token](#) for the messaging channel

Environment variables

Create a .env file in the project root. Example:

```
OPENAI_API_KEY=sk-...
DEFAULT_MODEL=gpt-4o-mini
```

```
# Optional - Telegram
TELEGRAM_BOT_TOKEN=
TELEGRAM_POLLING=true
```

```
# Usually leave as-is when using Docker Compose
DATABASE_URL=postgresql://orchestrator:orchestrator@postgres:5432/orchest
rator
REDIS_URL=redis://redis:6379/0
LOG_LEVEL=INFO
```

Variable	Required	Description
OPENAI_API_KEY	Yes (for real runs)	Used by ChatOpenAI in workflow and Telegram agents
DEFAULT_MODEL	No	Default gpt-4o-mini if an agent has no model set
TELEGRAM_BOT_TOKEN	No	Enables Telegram long polling in the worker
TELEGRAM_POLLING	No	Set false to disable Telegram while keeping the worker

Variable	Required	Description	
DATABASE_URL	/ No in Compose	Overridden	in for
REDIS_URL		docker-compose.yml containers	

First workflow run

1. Open the UI -> confirm seeded agents (Researcher, Writer, Planner, Executor, Reviewer).
2. **Workflows** -> open a template -> assign agents on nodes if needed -> **Save**.
3. **Runs** -> pick the workflow -> enter a task -> **Queue run**.
4. Open the run to watch steps/events; check **Messages** for the inter-agent transcript.

Makefile commands

```
make up      # docker compose up --build -d
make down    # stop all services
make logs    # follow container logs
make test    # pytest inside API container
make seed    # re-seed templates/agents (if DB empty)
make migrate # alembic upgrade head
```

Run without Docker (advanced)

```
# Terminal 1 - Postgres + Redis (or use compose for only data services)
docker compose up postgres redis -d

# Terminal 2 - API
cd backend
pip install -r requirements.txt
alembic upgrade head
python -m app.seed
uvicorn app.main:app --reload --port 8000

# Terminal 3 - worker
cd backend && python worker.py

# Terminal 4 - UI
cd frontend
npm install
VITE_API_URL=http://localhost:8000 npm run dev
```

Telegram setup

1. Create a bot with [@BotFather](#) and copy the token into .env as TELEGRAM_BOT_TOKEN.
2. make up - the worker starts long polling (works locally without webhooks).
3. Message your bot /start - it replies with your chat_id.
4. In the UI: **Agents** -> edit an agent -> **Channels** -> bind that chat_id.
5. Send text to the bot; replies appear in **Messages** (filter by Telegram).

Project structure

```
AI-Orchestrator/
backend/
```

```
app/
api/      # FastAPI routes
runtime/  # LangGraph compiler, executor, tools
channels/ # Telegram adapter
models/   # SQLAlchemy models
services/ # Queue, messages, scheduler
worker.py # Run queue + Telegram + schedules
alembic/  # Migrations
frontend/ # React + Vite + React Flow
docs/
architecture.md # Extension guide
DEMO.md # Recorded demo script
docker-compose.yml
Makefile
```

License

Rajaram - hiring challenge submission.